

Backend API Specification — Family Music Scrobbler PWA

Backend API Specification - Family Music Scrobbler PWA

A complete reference for all backend API endpoints exposed by the Family Music Scrobbler PWA. This specification defines authentication flows, data contracts, ingestion endpoints, and analytics routes used by the React PWA and background worker.

1. Overview

The backend is built using **FastAPI (Python 3.11+)** and exposes RESTful JSON endpoints. All endpoints follow:

- JSON request/response format
- OAuth2 Bearer token authentication (local auth)
- Spotify OAuth for ingestion
- Structured error responses

Base URL (development):

```
http://localhost:8000
```

Base URL (production):

```
https://<your-domain-or-ip>
```

2. Authentication API

POST /auth/register

Create a new local user.

Request

```
{  
  "email": "user@example.com",  
  "password": "string",  
  "display_name": "Andy"  
}
```

Response

```
{
  "id": "uuid",
  "email": "user@example.com",
  "display_name": "Andy"
}
```

POST /auth/login

Authenticate and receive a JWT access token.

Request

```
{
  "email": "user@example.com",
  "password": "string"
}
```

Response

```
{
  "access_token": "jwt-token",
  "token_type": "bearer"
}
```

GET /auth/me

Retrieve the authenticated user's profile.

Response

```
{
  "id": "uuid",
  "email": "user@example.com",
  "display_name": "Andy",
  "created_at": "2026-09-09T07:41:00Z"
}
```

3. Spotify OAuth API

GET /spotify/login

Redirects user to Spotify OAuth authorization.

GET /spotify/callback

Handles Spotify OAuth callback and stores encrypted refresh token.

Response

```
{
  "status": "linked",
  "user_id": "uuid"
}
```

4. Ingestion Worker API

GET /ingestion/status

Returns ingestion worker status.

Response

```
{
  "last_polled": "2026-09-09T07:30:00Z",
  "poll_interval_minutes": 10,
  "status": "running"
}
```

POST /ingestion/trigger

Manually trigger ingestion for the authenticated user.

Response

```
{
  "status": "started",
  "user_id": "uuid"
}
```

5. Tracks API

GET /tracks/recent

Retrieve recently played tracks for the authenticated user.

Query Params

- limit (default: 20)

Response

```
[
  {
    "track_id": "uuid",
    "name": "Song Title",
    "artist": "Artist Name",
    "played_at": "2026-09-09T07:20:00Z"
  }
]
```

****GET /tracks/**

Retrieve metadata for a specific track.

Response

```
{
  "track_id": "uuid",
  "name": "Song Title",
  "artist": "Artist Name",
  "album": "Album Name",
  "duration_ms": 210000
}
```

6. Leaderboard & Analytics API

GET /analytics/weekly-leaderboard

Returns weekly play counts per user.

Response

```
[
  {
    "user_id": "uuid",
    "display_name": "Andy",
    "play_count": 142
  }
]
```

GET /analytics/user-summary

Returns summary statistics for the authenticated user.

Response

```
{
  "total_scrobbles": 12345,
  "first_play": "2016-01-01T12:00:00Z",
  "top_artist": "Radiohead",
  "top_track": "Nude"
}
```

7. Scrobbles API

GET /scrobbles

Retrieve scrobbles with pagination.

Query Params

- limit (default: 50)
- offset (default: 0)
- from (ISO timestamp)
- to (ISO timestamp)

Response

```
[
  {
    "spotify_play_id": "string",
    "track_id": "uuid",
    "played_at": "2026-09-09T07:20:00Z"
  }
]
```

8. User Profile API

PATCH /users/me

Update profile settings.

Request

```
{
  "display_name": "New Name",
  "leaderboard_opt_out": true
}
```

Response

```
{
  "id": "uuid",
  "display_name": "New Name",
  "leaderboard_opt_out": true
}
```

DELETE /users/me

Delete user account and cascade delete scrobbles + tokens.

Response

```
{
  "status": "deleted",
  "user_id": "uuid"
}
```

9. Health & Diagnostics API

GET /healthz

Returns system health.

Response

```
{
  "database": "ok",
  "worker": "running",
  "api": "ok"
}
```

10. Error Response Format

All errors follow this structure:

```
{
  "detail": "Error message",
  "code": "error_code",
  "status": 400
}
```

Summary

This API specification defines all backend routes used by the Family Music Scrobbler PWA, including authentication, ingestion, analytics, and user management. It serves as the authoritative reference for frontend development, backend implementation, and planner-agent coordination.